Registro histórico de cambios del pipeline 3PL

1. Backup

Antes de realizar cualquier modificación se generó un **backup maestro del `azure-pipelines.yml` original**.

---

2. Configuración global

La configuración global se mantiene sin cambios:

```
trigger:
  - dev
  - main

pr:
  - dev
  - main

variables:
  vmImageName: "ubuntu-latest"
```

Esta sección define los triggers del pipeline y el agente utilizado. No pertenece específicamente a Cypress.

---

3. Build

El stage `Build` se mantiene dentro de `azure-pipelines.yml`.

Su responsabilidad continúa siendo generar el artefacto de la aplicación:

```
Código
  ↓
npm ci
  ↓
npm run build:dev / npm run build:prod
  ↓
dist
  ↓
vite-dist
```

El artefacto `vite-dist` posteriormente puede ser utilizado por el stage de Cypress y por los stages de Deploy.

---

4. Cypress

La configuración detallada de Cypress que anteriormente estaba dentro de `azure-pipelines.yml` fue extraída a:

```
templates/cypress-tests.yml
```

El stage `Test` permanece en `azure-pipelines.yml`, pero ahora referencia el template.

Para `dev`:

```yaml
- ${{ if eq(variables['Build.SourceBranch'], 'refs/heads/dev') }}:
    - template: templates/cypress-tests.yml
      parameters:
        useLocalBuild: true
        baseUrl: "http://localhost:8080"
        environment: "dev"
```

Para `main`:

```yaml
- ${{ if eq(variables['Build.SourceBranch'], 'refs/heads/main') }}:
    - template: templates/cypress-tests.yml
      parameters:
        useLocalBuild: true
        baseUrl: "http://localhost:8080"
        environment: "prod"
```

El stage continúa dependiendo del `Build`.

Responsabilidad del template

El template contiene la lógica común de Cypress:

```
Node.js
  ↓
Download vite-dist / iniciar aplicación local
  ↓
npm ci de automated-tests
  ↓
Configuración de variables
  ↓
npx cypress run
  ↓
Reports
  ↓
Screenshots
```

```
  ↓
PublishTestResults
```

El comportamiento de `Download vite-dist`, `serve` y `wait-on` depende del parámetro `useLocalBuild`.

---

5. Deploy DEV

El stage:

```
- stage: DeployDev
```

permanece en `azure-pipelines.yml`.

Su responsabilidad continúa siendo exclusivamente el deployment:

```
vite-dist
  ↓
Azure Static Web App DEV
```

La configuración de Cypress no contiene lógica relacionada con Azure Static Web Apps.

---

6. Deploy PROD

El stage:

```
- stage: DeployProd
```

también permanece en `azure-pipelines.yml`.

Su responsabilidad continúa siendo:

```
vite-dist
  ↓
Azure Static Web App PROD
```

---

7. Distribución final de responsabilidades

Elemento	`azure-pipelines.yml`	`cypress-tests.yml`	Motivo
`trigger` / `pr`	Sí	No	Control del pipeline
Build	Sí	No	Generación de la aplicación
`vite-dist`	Sí	Sí*	Build lo genera; Cypress lo consume cuando corresponde
`serve` / `wait-on`	No	Sí*	Solo necesarios cuando `useLocalBuild: true`
`npm ci` de Cypress	No	Sí	Configuración reutilizable
Variables de Cypress	No	Sí	Configuración reutilizable
`npx cypress run`	No	Sí	Núcleo de las pruebas
Reports	No	Sí	Reutilizable (Dentro del template)
Screenshots	No	Sí	Reutilizable
JUnit / `PublishTestResults`	No	Sí	Reutilizable
Deploy DEV	Sí	No	Delivery
Deploy PROD	Sí	No	Delivery

- El comportamiento depende del modo de ejecución.

---

8. Creación de `cypress-pipeline.yml`

Se creó un nuevo archivo en la raíz del proyecto:

```
cypress-pipeline.yml
```

Este pipeline está destinado exclusivamente a ejecuciones manuales de regresión.

```
trigger: none
pr: none
```

Esto significa:

- `trigger: none`: no se ejecuta automáticamente por Push.
- `pr: none`: no se ejecuta automáticamente por Pull Request.
- La ejecución se realiza manualmente desde Azure DevOps.

8.1 Selección de ambiente

El pipeline permite seleccionar el ambiente antes de ejecutar las pruebas:

```
parameters:
  - name: environment
    displayName: "Environment"
    type: string
    default: "dev"
    values:
      - dev
      - prod
```

Opciones disponibles:

```
dev
prod
```

8.2 Configuración del template para DEV

Cuando se selecciona `dev`, Azure incluye el template con:

```
useLocalBuild: false
baseUrl: "https://delightful-ground-005697610.5.azurestaticapps.net"
environment: "dev"
```

Esto significa que:

- No se descarga `vite-dist`.
- No se ejecuta `serve`.
- No se ejecuta `wait-on`.
- Cypress utiliza directamente la aplicación desplegada en DEV.

8.3 Configuración del template para PROD

Cuando se selecciona `prod`, Azure incluye el template con:

```
useLocalBuild: false
baseUrl: "https://app.3plicity.com"
environment: "prod"
```

El comportamiento es el mismo, pero Cypress apunta a PROD.

9. Parámetros del template

El template utiliza tres parámetros:

```
useLocalBuild
baseUrl
environment
```

useLocalBuild

Controla si Cypress debe preparar una aplicación local utilizando el artefacto generado por Build.

```
true
 ↓
Download vite-dist
 ↓
serve localhost:8080
 ↓
Cypress
```

o:

```
false
 ↓
No download
No serve
No wait-on
 ↓
Cypress contra aplicación desplegada
```

baseUrl

Define directamente la URL que utilizará Cypress.

En el pipeline normal:

```
http://localhost:8080
```

En el pipeline manual:

```
DEV → https://delightful-ground-005697610.5.azurestaticapps.net
PROD → https://app.3plicity.com
```

Por lo tanto, **el template no necesita determinar qué URL utilizar**. El pipeline que lo invoca proporciona el valor mediante el parámetro `baseUrl`.

`environment`

Indica al template si debe ejecutar la configuración de Cypress correspondiente a:

```
dev
```

o:

```
prod
```

---

10. Flujos finales

Pipeline normal — `azure-pipelines.yml`

```
Push / PR
    ↓
Build
    ↓
vite-dist
    ↓
Template Cypress
useLocalBuild = true
    ↓
Download vite-dist
    ↓
Install serve + wait-on
    ↓
Serve localhost:8080
    ↓
Wait for application
    ↓
Install Cypress dependencies
    ↓
Cypress DEV / PROD
    ↓
Reports + Screenshots + JUnit
```

```
    ↓
Deploy
```

Pipeline manual — `cypress-pipeline.yml`

```
Manual execution
    ↓
Seleccionar dev / prod
    ↓
Template Cypress
useLocalBuild = false
    ↓
No Download vite-dist
No serve
No wait-on
    ↓
Install Cypress dependencies
    ↓
Cypress contra URL desplegada
    ↓
Reports + Screenshots + JUnit
```

---

11. Arquitectura final

```
azure-pipelines.yml
│
├── Build
│    └── genera vite-dist
│
├── Test
│    ├── dev  → templates/cypress-tests.yml
│    └── main → templates/cypress-tests.yml
│
├── DeployDev
│    └── branch dev
│
└── DeployProd
     └── branch main


cypress-pipeline.yml
│
├── Selección DEV / PROD
│
└── templates/cypress-tests.yml
     └── Cypress → Reports
```

La lógica común de Cypress queda centralizada en:

```
templates/cypress-tests.yml
```

Mientras que `azure-pipelines.yml` mantiene la responsabilidad del **delivery** y `cypress-pipeline.yml` permite ejecutar **regresiones manuales**.

---

12. Uso del pipeline manual

El pipeline creado en Azure DevOps se denomina:

```
3PL Cypress Manual Regression
```

Para ejecutar una regresión:

1. Abrir **3PL Cypress Manual Regression** en Azure DevOps.
2. Seleccionar **Run**.
3. Seleccionar el ambiente:
    - `dev`

    - `prod`

4. Mantener las configuraciones establecidas.

5. Seleccionar **Run**.

El pipeline ejecutará Cypress contra la aplicación desplegada en el ambiente seleccionado y generará los reportes, screenshots y resultados JUnit correspondientes.

La configuración actual está basada en la rama:

```
F-tests
```
